

HelixSkills

HelixSkills

Sistema de habilidades respaldado por constitución y gobernanza para agentes de CLI AI.

Resumen

HelixSkills es un sistema de habilidades para agentes de CLI AI que incorpora el Helix Constitution como submódulo, por lo que todas las reglas universales de gobernanza se aplican de manera incondicional. Agrupa habilidades instalables para agentes, servidores de herramientas MCP, complementos de código Claude y motores reutilizables en un catálogo documentado y registrable.

Descripción breve

HelixSkills es un sistema de habilidades para agentes de CLI AI. Integra el Helix Constitution como submódulo para que todas las reglas universales se apliquen, e incluye habilidades registrables (prefijo de acción, validador de medios, multitrack, sincronización de sesión, ciclo de vida de elementos procesables y más), dos servidores de herramientas MCP, dos complementos de código Claude y motores reutilizables.

Descripción detallada

HelixSkills (repositorio skills, licencia Apache-2.0) es un sistema de habilidades para agentes de CLI AI, y parte de una inversión deliberada del orden habitual: primero la gobernanza, luego la capacidad. Hereda el Helix Constitution como submódulo constitution/, por lo que todas las reglas universales de constitution/CLAUDE.md y constitution/Constitution.md se aplican sin excepción —no como una convención que un agente pueda

respetar, sino como un conjunto de normas físicamente integrado en el árbol del proyecto. Un agente que adopta HelixSkills no puede eludir la constitución; las reglas viajan con el código.

Mientras que la mayoría de los “marcos de habilidades” operan con abstracciones, HelixSkills ofrece un inventario concreto y registrable que puede instalarse directamente. Siete habilidades constitucionales se instalan mediante `register.sh`: `action-prefix-system`, `media-validator`, `multitrack`, `reporting-workable-items`, `scheduled-work-queue`, `session-sync` y `workable-item-lifecycle` — un abanico que va de intermedio a avanzado, cubriendo desde la nomenclatura disciplinada de acciones hasta la validación de medios y el ciclo de vida completo de una unidad de trabajo. Además, ya están indexadas y preparadas habilidades en borrador (Android overview, lenguaje Java/Kotlin, sistema operativo Linux), a la espera de activación. Dos servidores de herramientas MCP (`media-validator` y `scheduled-work`) exponen estas habilidades a los agentes a través de Model Context Protocol, mientras que dos complementos de código Claude (`helix` y `scheduled-work`) integran las mismas capacidades directamente en el entorno de ejecución del agente —un mismo conjunto de habilidades, accesible para los agentes independientemente de la interfaz que utilicen.

Bajo el catálogo se encuentran cuatro motores reutilizables de profundidad 1 —continuum (implementado), junto con `session_orchestrator`, `token_optimizer` y `clickup_sync` (en diseño) —, la infraestructura compartida que evita que las habilidades reinventen la misma funcionalidad básica. El `token_optimizer`, por sí solo, declara un grafo de dependencias explícito que se extiende a los paquetes del ecosistema vasic-digital (TOON, Embeddings, VectorDB, Normalize, conversation) y al LLMProvider de HelixDevelopment, lo que permite auditar su integración entre repositorios en lugar de dejarla implícita. Todo el sistema está respaldado por una documentación rigurosa: un catálogo de habilidades, un índice autogenerated del grafo de habilidades, páginas de detalle por repositorio y un registro franco de Lagunas y Riesgos que identifica lo que aún no está completado. El sistema completo se replica en GitHub, GitLab, GitFlic y GitVerse para garantizar resiliencia y acceso regional.

Por qué lo creamos

Los agentes CLI y AI necesitan capacidades coherentes, reguladas y reutilizables, no scripts improvisados que reinventen las reglas en cada caso. HelixSkills se desarrolló para ofrecer a los agentes un conjunto de habilidades empaquetado y registrable, vinculado a una constitución compartida, de modo que el comportamiento se mantenga consistente y auditable en todos los agentes y proyectos que lo adopten.

Por qué cambia las reglas del juego

Hace que la capacidad de los agentes sea portátil y cumpla con las normas *por diseño*, no por disciplina. Cada habilidad es una unidad gobernada, versionada e instalable respaldada por un submódulo constitucional, de manera que, en el momento en que un agente registra una habilidad, hereda automáticamente el conjunto de reglas canónico, sin margen para desviaciones. Esto abre la puerta a algo que antes no era viable: trasladar una capacidad de un agente o proyecto a otro sabiendo que llega ya vinculada a la misma gobernanza, accesible a través de interfaces estándar (servidores MCP y complementos de código Claude), en lugar de un montón de scripts de integración ad hoc que reinventan las reglas en cada caso.

Qué tiene de innovador

- **Constitution como submódulo:** las reglas de gobernanza universales se heredan, no se copian. Se integran en el árbol de modo que cada agente que las consume queda sujeto al mismo conjunto de reglas canónico, con actualizaciones que fluyen desde una única fuente de verdad en lugar de una docena de copias obsoletas.
- **Habilidades entregadas como unidades auto-registrables** (`register.sh`) y ensambladas en un índice de grafo de habilidades generado automáticamente, para que el catálogo permanezca accesible y nunca pierda sincronía con lo realmente instalado.
- **Exposición multisuperficie:** el mismo conjunto de habilidades llega a los agentes a través de servidores de herramientas MCP y complementos de código Claude. Escribir una vez, comunicarse con el entorno de ejecución que utilice el agente.
- **Motores reutilizables de profundidad 1** (`continuum`, `token_optimizer`, `session_orchestrator`, `clickup_sync`) compartidos en todo el ecosistema, cada uno con declaraciones explícitas y auditables de dependencias entre repositorios, en lugar de acoplamientos ocultos.

Principales desafíos técnicos y cómo los resolvimos

- **Mantener el comportamiento de los agentes consistente y conforme a las reglas en múltiples habilidades y agentes:** reimplementar la gobernanza por habilidad garantiza divergencias con el tiempo. Se resolvió integrando el Helix Constitution como submódulo, de modo que las reglas en `constitution/CLAUDE.md` y `constitution/Constitution.md` se

apliquen de manera incondicional y se actualicen desde un único origen, en lugar de copiarse y quedar obsoletas.

- **Hacer que un conjunto de habilidades en crecimiento sea instalable y detectable:** un catálogo no sirve de nada si nadie puede encontrar o instalar lo que contiene. Se resolvió con el registro por habilidad mediante `register.sh`, que conecta cada una al instalarse, junto con un grafo de habilidades INDEX generado automáticamente y documentación detallada por repositorio, para que la detección refleje la realidad de forma automática.
- **Llegar a agentes que operan en distintos entornos de ejecución:** la misma capacidad no debería reconstruirse para cada host. Se resolvió empaquetando un único conjunto de habilidades tanto en definiciones de servidores de herramientas MCP (bajo `constitution/mcp/`) como en complementos de código Claude (bajo `constitution/plugins/`), de modo que una sola implementación esté disponible en todas las interfaces.

Pila tecnológica

- **Shell (lenguaje principal)** — elegido porque las herramientas de instalación y registro deben ejecutarse en cualquier entorno donde resida un agente, sin necesidad de un entorno de ejecución previo; impulsa `register.sh` e `install_upstreams`, manteniendo la incorporación libre de dependencias y portátil.
- **Submódulos de Git** — seleccionados para heredar la gobernanza sin duplicación: el Helix Constitution se monta en `constitution/` como referencia en vivo, de modo que las actualizaciones de reglas se propagan mediante un único puntero en lugar de copiarse y pegarse para luego olvidarse.
- **Model Context Protocol (MCP)** — adoptado como la interfaz estándar de herramientas para agentes, independiente del entorno de ejecución; se definen dos servidores MCP (`media-validator` y `scheduled-work`) bajo `constitution/mcp/` para exponer habilidades como herramientas invocables.
- **Complementos de código Claude** — elegidos para integrar habilidades de forma nativa en el entorno de ejecución del agente sin necesidad de código adhesivo; dos complementos (`helix` y `scheduled-work`) se distribuyen bajo `constitution/plugins/`, replicando la superficie MCP para un host diferente.
- **Motores reutilizables (`continuum`, `token_optimizer`, `session_orchestrator`, `clickup_sync`)** — seleccionados para extraer la maquinaria compartida de habilidades individuales y reutilizarla en distintos proyectos; `token_optimizer`, por ejemplo, está conectado a los paquetes `vasic-digital` (`TOON`, `Embeddings`, `VectorDB`, `Normalize`, `conversation`) y al `LLMProvider` de `HelixDevelopment` mediante dependencias declaradas, en lugar de duplicar código.

- **Espejado de Git en múltiples hosts (GitHub, GitLab, GitFlic, GitVerse)** — implementado para que la caída de un único host o un bloqueo regional no interrumpa el acceso; el mismo repositorio se mantiene activo en cuatro forjas para garantizar resiliencia y alcance.

Estado y notas de transparencia

- **Estado: beta.** Se han lanzado las siete habilidades constitucionales, dos servidores MCP y dos complementos; las habilidades en borrador están indexadas y pendientes de activación, y tres de los cuatro motores de profundidad 1 (session_orchestrator, token_optimizer y clickup_sync) aún están en fase de diseño.
- El archivo README se refiere al proyecto como helix_skills; la ruta canónica en GitHub es HelixDevelopment/skills. La cifra de hallazgos registrados en el README es un dato autoinformado.

Nivel de prioridad: Helix-principal.